Diagnosing and Mitigating TCP Recovery Costs During LEO Route Changes

Junhui Kim, Jio Lee, and Woojin Lim
KAIST CS30401 Network Lab 1
Team ID: 6

***Abstract*—LEO route changes can trigger TCP recovery through reordering or genuine loss. Using ns-3.48 and Hypatia, we distinguish a real-loss startup timeout from unnecessary recovery and compare preventive control, timestamp-based CUBIC restoration, and their combination. Across seven selected routes, the latest combination improves ten-second post-transition ACK rate by 0.053%–36.25%; whole-run gains for London, Manila, and Tokyo are 13.03%, 0.40%, and 0.010%. Fresh-state reproduction validates 46 deterministic cases: 35 improvement cases, seven baselines, and four queue cases. In London, restoration obtains 99.2% of the combination's incremental ACK-rate gain without an observed two-second p95 increase, but still requires a transition trigger. Restoration never activates in the latest combined cases. Manila fails the recorded method-relative delay limit despite being only 6.9% above default TCP. A queue sweep shows that throughput improvements can carry substantial SRTT costs. These results support conditional mechanisms with explicit deployment assumptions.**

I. Introduction

We study TCP CUBIC over changing satellite paths in the course implementation of Hypatia [1], [2]. Our concrete problem is a loss response triggered by reordering during a route transition, followed by reduced sending and slow recovery of throughput. Genuine queue loss may occur in the same transition, so treating every recovery event as a false alarm can make performance worse.

CUBIC is widely deployed, including adoption as the Linux default documented in RFC 9438 [3]. We retain the same variant across comparisons and use its exposed window and state traces to connect packet events with recovery. We investigate diagnosis, conditional mitigation, and the throughput–delay tradeoff; we do not establish that the phenomenon is unique to CUBIC.

II. Problem Statement

A. Startup timeout and missing drop coverage

In the Seoul-London q100 run, TCP enters the RTO handler at 1.831725262 s, transitions from RECOVERY to LOSS, and reduces its congestion window to one MSS. The original packet with sequence number 666541 is dropped at 0.609036964 s; its first retransmission is also dropped at 0.720677290 s. The timeout retransmission occurs at 1.831725262 s. This establishes a real-loss startup failure rather than inferring a timeout from the window plot alone.

Our original device-queue tracing missed Traffic Control drops that occur before a packet reaches the device queue. Adding this observation point corrected the earlier interpretation that the entire 30-second run was loss-free. The revised trace contains 185 TcDrop events. Two occur near 25.7 s, before the 26.2 s transition; no TcDrop occurs from 26 to 30 s. These earlier drops must not be assigned to the later unnecessary retransmission episode merely because the events are close in time.

The transition controllers do not establish a solution to this startup failure. We cross-check timeout-handler entry with congestion state and transmission behavior because handler entry alone need not produce a timeout response.

B. Reordering and mixed loss at route changes

At the London transition at 26.2 s, packet matching identifies 28 unnecessary retransmissions in the following two seconds, with no TcDrop in that window. The original q100 60-second cwnd trace separates two decreases. The first falls from a preceding value of 966 MSS to 676 MSS over approximately 25.822–25.855 s, after the two TcDrop events near 25.7 s. Fresh-run packet matching links the originals dropped at 25.711094579 and 25.766388990 s to retransmissions at 25.822035229 and 25.877337322 s, respectively, supporting real-loss recovery before the transition. The second falls from 696 MSS at 26.288190599 s to 487 MSS at 26.347518760 s, after the route change and within the TcDrop-free interval. Its ratio, 487/696 $\approx$ 0.6997, is consistent with CUBIC's multiplicative decrease factor β = 0.7 [3], also used by this ns-3 implementation. The separate value of 691 MSS is the sample immediately before the nominal 26.2-s transition; it is not the window immediately preceding the second decrease. At the New York transition at 48.9 s, 55 retransmissions contain 38 unnecessary retransmissions and 17 necessary loss recoveries. Its 18 TcDrop events include 17 originals and one unnecessary retransmission copy. These are packet classifications, not a subtraction of drop counts from retransmission counts.

The London baseline achieves 68.957 Mbps over 26.2–36.2 s, compared with 88.764 Mbps for v5 and 93.950 Mbps for Latest, both without retransmissions in that window. These controlled comparisons quantify the performance gap; they do not isolate the cost of retransmissions from the controllers' other effects. Fresh-state baseline and Latest window and SRTT traces are shown in Fig. 1.

A shorter new path allows packets to overtake packets still following the old path. TCP observes gaps and enters recovery, even when the originals eventually arrive. The performance

consequence includes reduced sending during recovery and altered CUBIC growth, not just the bandwidth consumed by duplicate copies. In New York, actual queue loss coexists with this effect. A useful mitigation must preserve real-loss handling while reducing the cost of reordering.

III. Proposed Ideas

A. Preventive route control

The v5 controller uses route geometry, link capacity, and queue observations to coordinate a transition. It retains valid old forwarding long enough for in-flight packets to drain, delays packets in directions where overtaking is predicted, and applies pacing based on estimated path and queue conditions. The protection buffer is sized from rate and waiting time, with burst allowance and a bounded memory budget. This extends our team's earlier fixed-duration and fixed-buffer prototypes to varying route conditions; these earlier prototypes are not an external prior-work baseline.

The guard_watch_undo combination changes when pacing starts: it enables pacing when estimated pressure warrants it, continues device-queue monitoring, and adds order protection. The latest queue_guard_undo revision also accounts for the order-protection buffer backlog when controlling pacing. This targets the immediate pacing restriction observed in London while retaining protection against loss and reordering elsewhere. The resulting controller is distinct from the original v5, so its gain cannot automatically be attributed to the restoration mechanism that is also enabled.

Deployment requires network support for forwarding retention and buffering, together with sender pacing control. The prototype has exact route forecasts and instantaneous queue information. An operational implementation could obtain forecasts from the operator that computes routing from orbital predictions and its own routing policy, then notify the sender of planned transitions. Public orbital data alone does not reveal the actual routing policy or queue state. Queue telemetry would need separate network measurements. The satellite operator would therefore need to provide route-change and telemetry information; delayed or inaccurate telemetry is not evaluated. Timing errors must fit within the overlap during which retained old links remain usable and must not invalidate the drain interval for in-flight packets. The 100-ms routing-update cadence is not a demonstrated 100-ms error tolerance: a one-update timing error could consume a substantial part of that overlap. Sensitivity to forecast and telemetry error remains untested.

B. Timestamp based restoration

Timestamp-based detection and response have established precedents in Eifel [4], [5]; our contribution is not the invention of timestamp-based undo. The experimental mechanism records pre-recovery state and checks timestamp echoes in ACKs against retransmitted packets. It restores state only when the required original-packet evidence is available. Mixed or unproven loss, timeout, or congestion indications can block restoration. The original implementation can restore window-related state as well as CUBIC growth state; the new comparison includes a CUBIC-only variant. These variants are kept separate in the evaluation.

Restoration operates after TCP has already reacted. It cannot prevent a queue drop or remove an already transmitted duplicate. Its target is the subsequent recovery cost. The experimental implementation is armed around a known transition, so it is not yet a fully autonomous sender-only policy. Timestamp evidence is available at the sender, but the current trigger still relies on transition knowledge.

C. Combining the mechanisms

Preventive control acts before recovery; restoration has an opportunity only if a qualifying recovery remains. Therefore, simply enabling both need not produce additive benefits. We compare the complete behavior, including activation counts, rather than treating an enabled feature as an effective feature. A city label or post-hoc knowledge of whether loss occurred is not an implementable online selection rule.

IV. Experiment Setup

We use the course Docker image 2026f-v1, ns-3.48, and the course Hypatia port [1], [2], [6]. The constellation has 1,584 satellites and shortest-distance routing updated every 100 ms. Both ISLs and GSLs use 100 Mbps links, following the course environment guide example [6]. One long-lived, 100 GB TCP flow uses CUBIC with a 1,380-byte MSS and seed 123456789. These fixed settings isolate the effects of queue size and transition control rather than capacity changes or competing flows.

The queue sweep uses 100, 250, 500, and 1,000 packets per queue over 60 seconds on Seoul-London. The guide example uses 1,000 packets; our 100-packet case exposes startup loss, and the 250- and 500-packet cases examine intermediate capacity. These are per-device queue limits, not an end-to-end buffer budget or a BDP-derived optimum. The baseline uses the same instrumented binary with all experimental interventions disabled. It retains default TCP behavior and dynamic routing; it is not a fixed-path control or an independently built stock binary. The latest controller comparison retains the diagnostic 100-packet setting and compares Seoul connections to London, New York, Sydney, Singapore, Los Angeles, Tokyo, and Manila. Tokyo exposed a delay regression and was subsequently used to develop the queue-aware correction. It is therefore a development repair, not an independent validation of that correction. Manila is the subsequent held-out route.

Throughput is cumulative newly acknowledged payload divided by the observation duration, expressed in Mbps. It is an ACK-based measure, not an independent receiver-delivered goodput measurement. For the latest transition comparison, throughput, retransmissions, drops, and RTOs use the ten seconds following the nominal transition. RTT p95 uses the first two seconds and the LastRTT trace. The common ten-second horizon follows the team's earlier comparison plan, covering immediate disruption and subsequent recovery. It is

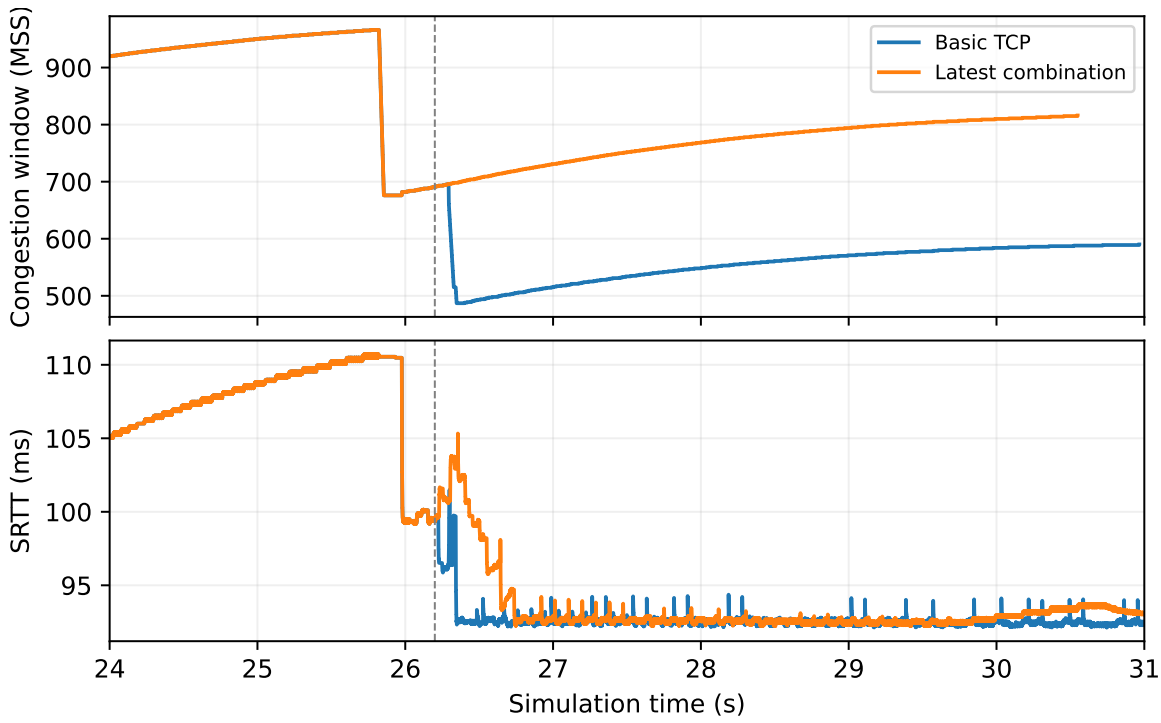


Fig. 1. Fresh-state London baseline and Latest around the 26.2-s transition (dashed line). Both show the earlier loss-recovery decrease; only baseline shows the subsequent reordering-related decrease. Packet matching links the two earlier drops to retransmissions, while the transition interval has no TcDrop. Latest preserves a larger cwnd but temporarily increases SRTT.

an event-local measurement. Whole-run loss and RTO counts are separate checks.

Queue analysis uses time-weighted SRTT, holding each estimator value until its next update. These quantiles describe time at an estimator value, not per-packet RTT or pure queueing delay. Earlier 5.5- and 10.5-second v5 measurements are not substituted into the ten-second comparison.

Re-executed denotes an actual run using regenerated routing inputs and the validated code package. All 46 cases completed: 35 improvement cases match 350 transport-trace hashes and 105 metric-window dictionaries; seven baselines pass inactive-control, configuration, pre-intervention prefix and reference-metric checks; four queue cases match their throughput/SRTT references. The two completed smoke cases were reused and rechecked in the full workflow. All presented runs are deterministic single-flow cases. Repeating the same seed verifies reproduction but does not create independent statistical samples. Instrumentation distinguishes device queues, Traffic Control drops, receiver arrival evidence, and TCP recovery state; duplicate observations across trace hooks are not added as independent drops.

V. Evaluation

A. Queue size trades throughput for delay

Table I summarizes the 60-second queue sweep. q250 attains approximately 98.5% of q1000 throughput while reducing median SRTT by 72.5 ms. q250 and q500 have similar whole-run throughput, but this includes their different startup histories. Over 20-60 s, q500 and q1000 both reach approximately 96.233 Mbps, while q500 has a 34.1 ms lower median SRTT. This supports an intermediate queue in this scenario, not a universal optimal queue size. Figure 2 visualizes this tradeoff.

TABLE I
Seoul–London queue sweep. ACK rates are Mbps; SRTT medians are time-weighted milliseconds.

Queue	0–60 s		20–60 s	
(packets)	ACK	SRTT	ACK	SRTT
100	67.313	96.108	84.392	92.470
250	93.602	102.834	94.628	100.278
500	93.579	133.549	96.233	132.907
1000	95.006	175.337	96.233	167.029

B. Restoration helps when its evidence permits

The matched CUBIC-only restoration arm increases London ten-second ACK throughput from 68.957 to 93.753 Mbps (+35.96%). It still transmits the same 28 retransmissions as baseline: restoration addresses the growth-state penalty after recovery rather than preventing the duplicate transmissions. New York, Sydney, and Singapore obtain the same ACK rate as their baselines. The controller evidence distinguishes rejection in New York and Sydney (one recovery attempt, zero applications each) from no restoration attempt in Singapore. Equality of throughput alone cannot identify the reason.

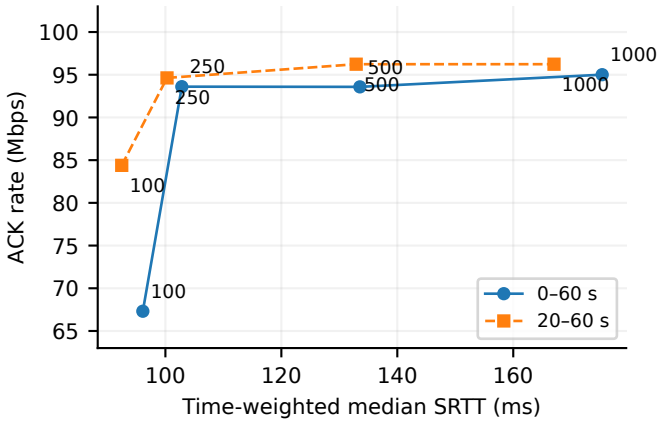


Fig. 2. Queue throughput–delay tradeoff in the re-executed Seoul–London sweep. Labels denote queue capacities in packets. The 20–60-s view separates later operation from startup.

TABLE II
TEN-SECOND ACK THROUGHPUT (MBPS). LATEST IS QUEUE_GUARD_UNDO. ALL ENTRIES WERE RE-EXECUTED USING REGENERATED ROUTING INPUTS.

Route	Base	v5	Undo	Latest
London	68.957	88.764	93.753	93.950
New York	90.564	96.103	90.564	96.103
Sydney	92.039	96.101	92.039	96.101
Singapore	81.499	95.991	81.499	95.991
Los Angeles	90.262	95.922	94.020	95.955
Tokyo	94.406	94.460	94.418	94.456
Manila	92.490	95.807	90.686	95.814

Manila provides a negative result: CUBIC-only restoration obtains 90.686 Mbps versus baseline's 92.490 Mbps (-1.95%). The re-executed evidence records one restoration application at 80.961176729 s. It restores CUBIC state while leaving cwnd at 262,200 bytes (190 MSS) before and after application. This excludes an immediate cwnd jump as the explanation, but does not establish why subsequent ACK throughput decreases. The causal mechanism remains unresolved; queue overflow after restoration is an unverified hypothesis.

C. Revised preventive control and the combination

Table II compares re-executed baselines with the three methods. London Latest improves ten-second ACK rate by 36.25% and raises two-second LastRTT p95 from 96 to 102 ms. Manila gains 3.59%, approximately matching v5. Tokyo gains only 0.053%, despite eliminating 63 retransmissions and reducing LastRTT p95 from 21 to 15 ms. Avoiding retransmissions can yield little throughput gain on an already well-utilized path. Figure 3 compares the same event-local ACK rates.

The latest combination reports zero retransmissions, Traffic Control drops, protection-buffer drops, and RTOs in these seven transition windows. Restoration activates zero times. The result demonstrates preventive control behavior, not an added restoration benefit. Whole-run loss and startup RTOs remain. For example, the re-executed London latest run has

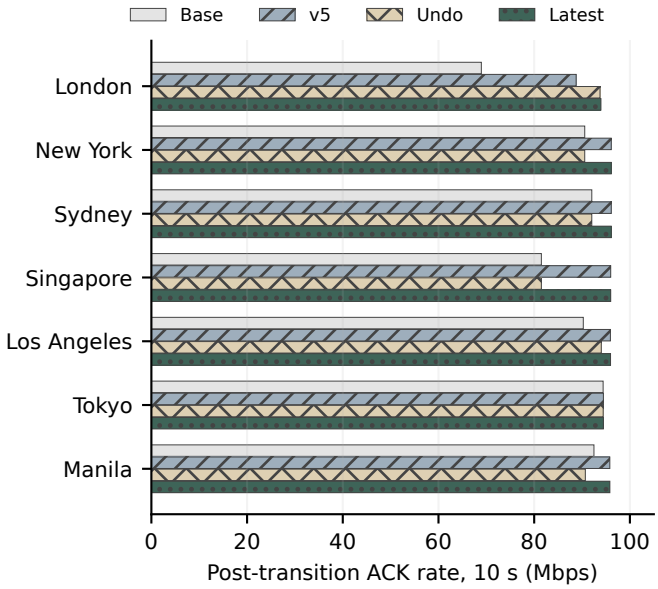


Fig. 3. Ten-second ACK-rate comparison using Table II. All seven baselines and all improvement entries were re-executed using regenerated routing inputs.

185 TcDrop events and one RTO over 0-36.2 s, despite zero in the transition window. The 185 count is present in this arm's whole-run result and, separately, in the original 30-s diagnostic drop log; equality of counts alone does not establish identical dropped packets.

The range of latest transition gains across all seven rows is 0.053%–36.25%. Whole-run gains for London (0–36.2 s), Manila (0–90.9 s), and Tokyo (0–51.8 s) are 13.03%, 0.40%, and 0.010%. These city-specific durations are not a common whole-run observation window.

D. Queue-aware repair and a held-out delay limit

The preceding guard_watch_undo controller eliminates Tokyo's 34 retransmissions from a configuration without order protection but increases its two-second LastRTT p95 from 15 to 21 ms. Original v5 has 18 ms. After observing this failure, queue_guard_undo adds protection-buffer backlog to pacing monitoring. In Tokyo it reduces p95 to 15 ms while retaining zero transition retransmissions and a ten-second rate of 94.456 Mbps. This is a successful repair on an observed failure, not independent evidence of generalization.

In Manila, the older guard_watch_undo achieves 86.056 Mbps with two protection-buffer drops. Latest reaches 95.814 Mbps with no buffer drops or retransmissions, approximately matching v5 at 95.807 Mbps. Its two-second LastRTT p95 is 31 ms. The team's pre-performance selection workflow records a limit of 110% of both v5 and auto_watch_undo, giving a binding threshold of 30.8 ms; Latest therefore fails. Against default TCP's 29 ms, the increase is 2 ms (6.9%), within a separate descriptive 10% baseline-relative bound. LastRTT has integer-millisecond resolution, but we retain the original criterion and failure. The README preserves the criterion's provenance and audit limitations.

TABLE III
LONDON DEPLOYMENT TRADEOFF. ACK CHANGE IS RELATIVE TO BASELINE OVER 26.2–36.2 S; LASTRTT P95 USES 26.2–28.2 S (BASELINE: 96 MS).

	CUBIC-only Undo	Latest
Deployment Requirements	Sender Transition trigger; timestamp evidence; saved CUBIC state	Operator and sender Route forecasts; forwarding retention; protection buffer; queue telemetry; pacing
ACK change	+35.96%	+36.25%
LastRTT p95	96 ms	102 ms

E. Deployment tradeoffs and scope

In London, CUBIC-only restoration captures (93.753-68.957)/(93.950-68.957) = 99.2% of the latest method's incremental ACK-rate gain. Its two-second LastRTT p95 stays at 96 ms, whereas the latest combination reaches 102 ms. Restoration leaves the 28 retransmissions intact, while preventive control eliminates them. Thus, near-equal throughput does not imply identical packet behavior. Table III summarizes the deployment requirements and observed London tradeoff.

The restoration logic changes sender state and needs no router retention or protection buffer; nevertheless, the current prototype is armed with externally known transition timing. It is not demonstrated as a fully autonomous sender-only deployment. The latest controller additionally requires forwarding retention, protection buffering, queue telemetry, and pacing coordination. These resource and coordination requirements are qualitative costs: we have not measured production CPU overhead, signaling load, or total memory expense. In Los Angeles, restoration improves ACK throughput by 4.16% and Latest by 6.31%; New York, Sydney, and Singapore show no restoration gain but Latest gains 6.12%, 4.41%, and 17.78%. These selected cases illustrate broader preventive benefits, not effectiveness on every route. Manila's negative restoration result and failed combined delay criterion preclude a universal recommendation.

The study uses one saturated flow, one seed, exact route forecasts, and instantaneous telemetry. It does not establish fairness or robustness to forecast error. Contemporary loss detectors such as RACK-TLP explicitly address reordering [7]; these results do not establish the same behavior under a modern Linux RACK-TLP stack. All 46 cases were validated after correcting two input-generation helper issues. The official course patch also applied and built in a second clean ns-3 volume with matching final source hashes; details are in the README. These checks establish reproducibility, not independent statistical replication. Startup RTO prevention and Manila's restoration slowdown remain unresolved.

VI. CONCLUSION

Packet-level evidence distinguishes genuine startup loss from unnecessary transition recovery. Larger queues improve throughput at an SRTT cost; preventive control and timestamp-based restoration address different parts of recovery and do not automatically provide additive gains. Latest improves London's transition ACK rate by 36.25%, but Tokyo's gain is only 0.053%, and its seven cases show no incremental restoration benefit. Queue-aware pacing repairs Tokyo's observed regression while Manila still fails the recorded delay limit. A deployment decision should therefore weigh packet behavior, throughput, delay, and required network support, retaining negative results.

REFERENCES

[1] S. Kassing, D. Bhattacherjee, A. B. Aguas, J. E. Saethre, and A. Singla, "Exploring the Internet from space with Hypatia," Proceedings of the ACM Internet Measurement Conference (IMC), 2020.

[2] KAIST CS30401, "Network Lab 1 Improving TCP for LEO Satellite Networks," course handout, Fall 2026.

[3] L. Xu, S. Ha, I. Rhee, V. Goel, and L. Eggert, Ed., "CUBIC for Fast and Long-Distance Networks," RFC 9438, Aug. 2023.

[4] R. Ludwig and M. Meyer, "The Eifel Detection Algorithm for TCP," RFC 3522, Apr. 2003.

[5] R. Ludwig and A. Gurtov, "The Eifel Response Algorithm for TCP," RFC 4015, Feb. 2005.

[6] KAIST CS30401, "Lab 1 Environment Guide," course environment documentation, Fall 2026.

[7] Y. Cheng, N. Cardwell, N. Dukkipati, and P. Jha, "The RACK-TLP Loss Detection Algorithm for TCP," RFC 8985, Feb. 2021.